Assignment - 3.

PRATHAM L KAMATH
05109013001H.

1] Analyse how a race condition can occur when multiple kernel threads access a shared device buffer without proper synchronization. Illustrate with an example from a typical device driver scenario & propose a method to prevent it.

= Race condition occurs when 2 or more kernel threads or interrupt handlers access shared data at the same time & the final result depends on the order in which the access occur.

Ex:- In the scull driver, memory is allocated dynamically when data is written to the device.

The code that check's whether memory has already been allocated looks like this:

```
if (!dptr->data[s_pos]){
        dptr->data[s_pos] = kmalloc(quantum, GFP_KERNEL);
        if (!dptr->data[s_pos])
                goto out;
}
```

Here dptr->data[s_pos] is a pointer to a part of the devile's memory buffer. Before writing, the driver checks if this memory block has been allocated. If not, it allocates memory using kmalloc() & assigns the pointer.

Now imagine Proceses A & B both trying to write to the same part of the scull devile at the same time:

struct scull_qset *data;
int quantum;
int qset;

1. Both processes reach the if(!dptr->data[s_pos]) check together. The pointer is currently NULL for both.
2. Both decide to allocate memory using kmalloc(). Each process gets a different memory block.
3. Both then assign their pointer to dptr->data[s_pos]
4. The process that finishes 2nd overwrites the pointer of first.

The memory allocated by the 1st process is lost.

The memory allocated by Process A is never freed. This causes a memory leak in kernel.

In more serious cases it could lead to Data corruption, system crashes, security vulnerabilities etc. Programmers might think, this situation is very rare, so it won't happen. But in real systems such rare events happen very frequently, especially on multi core processors because many threads run in parallel. Even one in a million chance can occur many times per second in the kernel.

One solution to this problem is to use lock that excludes all concurrent access paths that can touch the shared entity. which in this case is a shared device buffer.
One such lock is semaphore.

In the scull driver, a semaphore is used to protect the scull_dev data structure, which contains all the important information for each device.
Each scull device has it's own data structure:
struct scull_dev {

```
struct scull_qset *data;
int quantum;
int qset;
unsigned long size;
unsigned int access_key;
struct semaphore sem;
struct cdev cdev;
};
```

The sem field is the semaphore that protects the rest of the structure. Each scull device has it's own semaphore so that processes accessing different devices can run in parallel.

Before any process can use the scull device, its semaphore must be initialized. This happens during module loading:

```
for (i=0; i< scull_nr_devs; i++) {
    scull_devices[i].quantum = scull_quantum;
    scull_devices[i].qset = scull_qset;
    init_MUTEX(&scull_devices[i].sem);
    scull_setup_cdev(&scull_devices[i], i);
}
```

init_MUTEX() initializes the semaphore & it is done before the device is registered (scull_setup_cdev())

Whenever the driver accesses shared data, like in scull_write(), it locks the semaphore before entering the critical section

```
if (down_interruptible(&dev->sem))
    return -ERESTARTSYS;
```

Releasing the lock: At the end of the function or if an error occurs, semaphore must be released.

```
Out: up(&dev->sem); // up() releases the semaphore.
     return retval;
```

Thus semaphore prevents 2 or more processes from writing to the same device, data at the same time & avoids memory leaks & data corruption

2] Evaluate the advantages & limitations of using atomic operations compared to spinlocks for protecting shared data in an interrupt context. Under what circumstance would one mechanism be preferred over the other? Justify with reasoning based on kernel behaviour.

= In the Linux kernel, when multiple threads or interrupts try to update shared data, proper synchronization is needed to avoid race conditions. 2 common methods: Atomic operations & Spinlock

(i) Atomic operations:

Atomic operations are low level, hardware supported operations that ensure a single instruction executes completely without interruption. They are used to safely modify single variables shared blw threads or interrupt handlers.

Ex:

```
atomic_t counter;
atomic_inc(&counter);
atomic_dec(&counter);
atomic_add(5, &counter);
```

Advantages:

(i) Fast & lightweight - No need to acquire or release locks
(ii) Non-blocking - Does not cause sleep or context switches.
(iii) Safe in interrupt context.
(iv) Ideal for simple shared data.

Limitations:

(i) Works only on single integer sized variables.
(ii) Cannot maintain relationship b/w multiple variables
(iii) No way to ensure mutual exclusion across a group of operations
(iv) On multiprocessor systems, heavy contention on atomics may still cause performance degradation due to cache line bouncing.

(ii) Spinlocks.

A spinlock is a kernel locking primitive that allows only one processor to enter a critical section at a time. If a CPU tries to acquire a lock already held by another it will spin in a tight loop until the lock is free.

Eg:
```
spinlock_t my_lock;
spin_lock_init(&my_lock);
spin_lock(&my_lock);
spin_unlock(&my_lock);
```

Advantages:

(i) Protects complex or multiple shared variables.
(ii) Provides mutual exclusion for critical section.
(iii) Safe in interrupt contexts.
(iv) Works efficiently for short, quick operation.

Limitations:

(i) CPU busy-waits while spinning - wastes CPU cycles.
(ii) Must not be used in code that can sleep.
(iii) Not suitable for long running or blocking operation.

Use atomic operation when we only modify a single integer or flag. When we need to perform with minimal overhead. The operation is simple.

Use spinlocks when we need to protect multiple variables or a data structure. The code is non-sleeping. Several operations must be executed as a single atomic unit.

3] In SMP systems, memory operations may not execute in Program order. Analyze how memory barriers enforce ordering & prevent subtle race conditions in driver code. Provide an ex. illustrating the consequences of missing a barrier.

= A compiler can cache data value into CPU register & without them to memory & even if it stores them, both write & read operations can operate on cache memory without ever reading RAM. In SMP systems, CPU can reorder memory operations for Performance reasons. This can lead to subtle race conditions when multiple processors share data because one CPU may see memory changes in a different order than other.

Device drivers often communicate with hardware registers & shared buffers & also depend on a specific order of reads & writes. If these operations are reordered, a drives may:

- Signal a drive to start before all data is written.
- Cause data corruption etc.

Solution to this is Memory Barriers.

A memory barrier is a special instruction that forces the CPU to complete all pending memory operations before continuing.

Types of memory barriers in Linux:
defined in <asm/system.h> or <linux/kernel.h>

barrier() → Prevents compiler from reordering operations, no hardware effect.

rmb() → Ensure all reads before it complete before later reads.

wmb() → Ensures all writes before it complete before later writes.

mb(), read_barrier_depends(), smp_rmb(), smp_wmb(), smp_mb()

They make the CPU & compiler respect the intended order of execution, ensuring that shared variables, device registers or I/O memory are updated in the right sequence.

Ex:- Missing memory Barrier.

dev->registers.addr = io_addr;
dev->registers.size = io_size;
dev->registers.operation = DEV_READ;
dev->registers.control = DEV_GO;

If the CPU reorders these writes, DEV_GO might be issued before the address & size registers are updated. The device could then read the wrong address or size causing incomplete transfer, corrupted data, device crashes etc.

These flags function like

4] Examine how the slab allocator optimizes kernel memory usage through object caching & reuse. Analyze the trade offs b/w using the slab allocator & traditional allocation methods.

= The slab allocator manages caches of pre-constructed objects of the same type & size. Each cache is made up of one or more contiguous slabs which contains several objects. When a driver requests a frequently used object, the allocator:

1. Checks the object cache.
2. If a free object exists, it returns it immediately.
3. If not, it allocates a new slab & divides it into multiple objects.

When an object is freed, it is returned to the cache instead of releasing it to the system ready for reuse. This avoids the cost of repeated allocation & initialization.

The Tradeoffs include:

1. High setup overhead: compared to other methods, creating a dedicated cache using kmem_cache_create uses more memory initially.
2. Possible wasted space. Each slab must hold whole objects & if only few of them are used there are possibilities of space wastage.
3. Less flexible for large buffers - For large, one-off allocations, kmalloc() or vmalloc() are more suitable.

④ Complex internal management: Each cache maintains its own metadata & free lists thus slightly increasing kernel memory footprint.

⑤ No repeated construction/destruction: Traditional allocation methods like kmalloc() or __get_free_pages() allocate raw memory blocks which lead to constructing & destructing similar objects repeatedly.

⑥ Avoids fragmentation & Poor CPU cache locality caused by using traditional allocation methods.

⑤ Evaluate how the choice of GFP flags (eg. GFP_KERNEL, GFP_ATOMIC, GFP_DMA) affects allocation behaviour under various context such as interrupt handler, process context & DMA operations. Analyze a scenario in which using an incorrect flag result in an allocation failure.

= GFP_KERNEL:
Normal allocation of kernel memory. May sleep.

GFP_USER:
Used to allocate memory for user-space pages. it may sleep.

GFP_HIGHUSER:
Like GFP_USER, but allocates from high memory if any. (High memory is described in the next section)

GFP_NOIO
GFP_NOFS:
These flags function like GFP_KERNEL, but they

add restrictions on what the kernel can do to satisfy the request. A GFP_NOFS is not allowed to perform any filesystem calls, while GFP_NOIO disallows the initiation of any I/O at all. They are used primarily in the filesystem & virtual memory code where an allocation may be allowed to sleep, but recursive filesystem calls would be a bad idea

--GFP_DMA:
This flag requests allocation to happen in the DMA-capable memory zone. The exact meaning is platform dependent.

--GFP_HIGHMEM
This flag indicate that the allocated memory may be located in high memory

--GFP_NOWARN
This rarely used flag prevents the kernel from issuing warnings when an allocation cannot be satisfied.

--GFP_HIGH
This flag marks a high priority request which is allowed to consume even the last pages of memory set aside by the kernel for emergencies.

Ex for incorrect flag usage: In context where kMalloc is called from outside a process's context, GFP_KERNEL isn't always the right allocation flag to use. This can happen in interrupt handlers, task queues, kernel timers etc.

Since GFP_KERNEL may put the current process to sleep & this should not happen in the above scenarios, as it would lead to allocation failure. GFP_ATOMIC can be used instead since it never puts the process to sleep & allows kmalloc to use the free pages left out by the kernel to fulfill Atomic allocation.

6] Analyze the difference b/w __get_free_pages() & kmalloc() in terms of memory allocation granularity, physical contiguity & performance. In what situations would one be preferable over the other & why?

__get_free_pages():

(i) Memory Allocation - Allocates contiguous pages in block of size PAGE_SIZE x 2 order.

(ii) Granularity - Granularity is in units of pages rather than objects.

(iii) Physical contiguity - Returns physically contiguous pages, directly from the buddy system allocator. Larger contiguous regions are harder to allocate under memory pressure.

(iv) Performance - Slightly slower than kmalloc() because it works directly with the buddy system. ~~Allocates Larger~~ Performance for higher orders can

due to fragmentation & search overhead

(v) Preferable when - multiple contiguous pages are needed like in mmap() buffers.

- For page based allocations
- When the memory will be directly mapped into user space or used for low level memory manipulation.

kmalloc():

(i) Memory allocation: Allocates memory in power of 2 size classes managed by the slab allocator.

(ii) Granularity: It is based on object size, rounded up to next cache size.

(iii) Physical contiguity: Returns physically contiguous memory suitable for DMA. Allocation is fulfilled from pre defined slab caches.

(iv) Performance: Very fast since it uses slab caches, so allocations are often O(1) & reuse pre allocated memory chunks. Lower fragmentation for small objects.

(v) Preferable when small to medium allocations frequently allocated / free kernel buffers.

- When cache alignment & reuse matter.

4] Analyze the differences blw IP Port mapped & memory-mapped I/O mechanisms in the Linux kernel. How do these affect driver performance, portability & hardware interactions? Provide an example of when each method is appropriate.

= In the Linux kernel, device registers can be accessed in 2 ways - I/O port mapped I/O & memory mapped I/O. In port mapped I/O, device uses a separate I/O address space & CPU communicates with them using special instructions like 'in' & 'out'. Functions such as inb() & outb() are used in kernel code to perform these operations. This method is mainly used on x86 architecture for legacy drivers such as serial or parallel port.

In contrast, memory mapped I/O maps device registers into the system's main memory address space, allowing the CPU to access them using regular memory read/write instructions. The kernel provides normal CPU memory operations, benefiting from caching & pipelining & works on all modern architecture making it more portable than Port mapped I/O.

Performance wise MMIO is generally more efficient, while Port mapped I/O offers better compatibility with older x86 architecture. Port mapped I/O is suited for legacy ISA device & MMIO for modern high speed service due to its better speed & portability.

Ex:- I/O Port mapped I/O

The PC parallel port or legacy serial port uses I/O Port address.

```
Outb (value, 0x378);
data = inb (0x379);
```

Memory mapped I/O:

```
void __iomem *mmio_base = ioremap (0xF0000000, 0x100);
iowrite32 (value, mmio_base + REG_CONTROL);
```

8] Compare & analyze polling based & interrupt driven methods for communicating with hardware devices. Under what conditions would Polling be more efficient than interrupts & vice versa? Justify example for device driver Operations.

= In Linux device drivers, communication with hardware can be handled using either polling or interrupt driven I/O. In the polling method, the CPU repeatedly checks (polls) a device's status register to see if it has completed an operation or needed attention. This approach is simple but inefficient for most situations as it keeps the CPU busy waiting for events wasting Processing time that could be used for other tasks. However polling has very low latency since the CPU can immediately detect the devices status, making it useful for fast or Predictable hardware

Where events occur frequently or at regular intervals such as during device initialization, real time control systems or simple devices that complete operations quickly.

In contrast interrupt driven I/O allows the device itself to signal the CPU when it requires attention by triggering an IRQ. When the interrupt occurs the kernel temporarily stops the current process, executes the device's interrupt handler & then resumes normal execution. This method is much more efficient for devices that generate unpredictable events like — keyboards, network cards or disk — controllers because the CPU can perform other work while waiting. Interrupts reduce CPU load & improve overall system responsiveness but introduce slightly higher latency due to the overhead of context switching & interrupt handling.

Polling is best when device events occur rapidly or predictable as it avoids the cost of frequent interrupts. It is often used in short data transfers or real time applications that require immediate response.

Interrupt driven I/O is more reliable for slow or irregular devices where continuously polling would waste CPU cycles. Ex: A network interface card or keyboard driver uses interrupts to handle sporadic events efficiently.